

Make Graphs Great Again: Rethinking LLM Agent Architecture via Structure-Semantic Separation

Paper #1705

Abstract

The generality of LLMs makes them a powerful abstraction for agent design. It naturally encourages an **LLM-centric agent architecture** in which each observation is routed back to the LLM for the next decision. However, this convenience hides an architectural drawback: the model must repeatedly reread growing context, preserve task state in prompt history, and spend expensive online inference even on routine execution progress. As a result, the cost of existing LLM agents is usually extremely high.

We present **structure-semantic separation** as an architectural boundary for LLM agents. Instead of letting the LLM act as the default executor for every control step, this boundary replaces repeated model control with explicit structural execution whenever progress can be verified by the runtime, while reserving the LLM for semantic execution that structure cannot resolve. We implement this boundary in MotifAgent with motif graphs: the Structure Engine executes verified graph progress, while the Semantic Engine handles unresolved semantic gaps through scoped LLM calls. Moreover, exposing structural information to the serving layer enables two system optimizations: Structural-Aware (SA) scheduling and fine-grained KV-cache retention. Experimental results show that this separation significantly reduces unnecessary model invocation and token consumption.

Keywords: LLM agents, agent runtime, LLM serving, KV cache

1 Introduction

Enabled by the powerful abilities of Large Language Models (LLMs), LLM agents [8, 14, 20, 24, 26, 28] are increasingly used in various domains such as coding, web automation, and personal assistance. Because LLMs can flexibly interpret new observations and formulate action decisions in natural language, an LLM-centric agent architecture has become a natural default: after each observation, control is returned to the LLM, which decides the next step. This design preserves the generality and flexibility of LLMs, but it also makes the core control logic of the agent program highly dependent on LLM calls.

This convenience is not free. Each decision step introduces an expensive LLM call with growing context. The high cost of this architecture has motivated many efforts to accelerate LLM agents, such as batching, scheduling, and KV-cache management [6, 29, 32]. These techniques are good, but most of them still optimize within the LLM-centric loop. They

make individual model-mediated steps cheaper and faster, but the agent often still returns to the LLM after each observation. As a result, the execution progress itself remains a recurring inference workload.

We argue that this bottleneck is architectural rather than merely computational. In many agent executions, especially after the initial ambiguity has been resolved, the next step is not always an open-ended semantic decision. It may follow a stable tool order, transfer a value from one tool result to a later input, release a downstream operation after its dependency is satisfied, or check a known precondition. Under an LLM-centric architecture, these structural transitions remain encoded in prompt text rather than executable runtime state. The runtime observes tool calls and tool results, but it does not own the recurring structure that connects them.

This opportunity is especially clear because tool use has become a standard execution mechanism for modern LLM agents. Tool calls expose explicit runtime boundaries, including tool names, input schemas, returned fields, and parameter dependencies across calls. These signals make the recurring execution structure observable to the runtime, unlike consecutive model-only reasoning steps whose boundaries depend heavily on prompting choices and model behavior. The missing piece is a **redesign of control ownership**: the runtime should advance structural progress when current evidence is sufficient, while the LLM should be reserved for semantic gaps that structure cannot resolve.

Figure 1 illustrates this boundary with a refund task. Figure 1(a) shows the LLM-centric execution, where the request “refund the item I bought last Friday” is handled as a sequence of model-mediated tool steps. After each tool result, the agent returns to the model to decide whether to search records, check eligibility, submit the refund, or stop. Figure 1(b) shows the separated execution. The initial reference to “the item I bought last Friday” may still require semantic mediation because the target object is ambiguous. After the relevant record is identified, the remaining execution can follow a runtime-owned motif graph. The runtime can search records, extract the item or order identifier, check refund eligibility, and submit the update. These steps become structural when the runtime has enough evidence to determine the next tool arguments and confirm that the relevant preconditions are satisfied. If required information is missing, or if an unexpected eligibility result invalidates the current path, execution returns to the LLM as a semantic

(a) Model-centric execution: every tool result returns to the LLM



(b) Structure-semantic separation: runtime owns structural progress

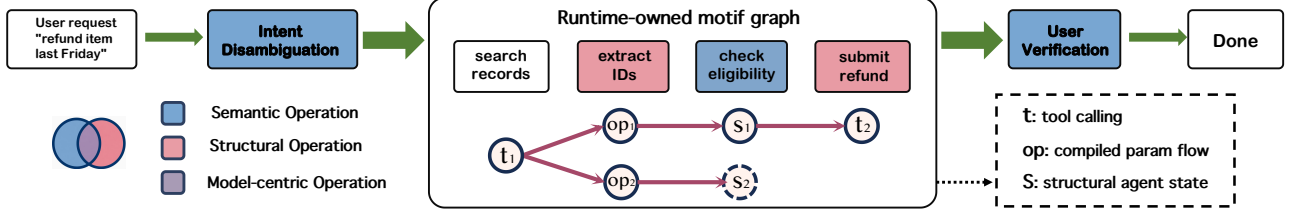


Figure 1. Running example for a refund task. (a) In LLM-centric execution, every tool result routes control back to the LLM. (b) Under structure-semantic separation, semantic operations such as intent disambiguation remain LLM-mediated, while stable tool order and parameter flow are owned by the runtime as a motif graph.

gap. The returned prompt is also narrower. Rather than asking the model to reconstruct the entire execution history, it is scoped by the current motif state and only contains the evidence needed to resolve the gap. This example shows that structure is not a label that is assigned to the whole task in advance. It is a run-time property of each transition. We call this boundary structure-semantic separation.

We build MotifAgent, an **agent-runtime and serving co-design system** that realizes structure-semantic separation. MotifAgent stores recurring tool-use knowledge in a motif library with two complementary forms. *Positive motifs* are executable local workflows that encode tool operations, parameter flow, dependencies, and the runtime evidence required to validate each transition. When a positive motif is matched, the Structure Engine instantiates it as a runtime object and advances graph execution only after the required information and preconditions have been satisfied. *Negative motifs*, in contrast, are non-executable guard patterns extracted from failed or abandoned trajectories. They monitor ReAct fallback execution and detect repeated terminal failure patterns before the agent spends more model calls. When execution reaches a semantic gap, the runtime sends a typed mediation request to the Semantic Engine, which invokes the LLM for bounded tasks such as slot filling, candidate selection, or clarification. Motif knowledge is therefore not a fixed whole-task workflow: positive motifs drive validated structural execution, while negative motifs guard ReAct fallback against repeated terminal failures.

Runtime-owned motifs also change what the serving system can observe. In an LLM-centric agent loop, the backend mostly sees LLM calls that have reached the serving layer, together with token-level features such as prefixes and lengths. It has little visibility into the local tool workflow behind each call or the downstream operations that the call may release. With motif graphs, the runtime exposes motif stage,

structured prompt prefix, and possible downstream releases. This turns agent structure into a serving-visible signal. We use these signals for structure-aware scheduling and fine-grained KV-cache retention and eviction.

Our evaluation studies both the agent-runtime and serving implications of structure-semantic separation on representative tool-use agent workloads, including APIBank [8], TauBench [27], and AppWorld [18]. Compared with ReAct-style executions, MotifAgent reduces the number of LLM calls by up to 27% and token consumption by up to 36% through motif skills while preserving task accuracy. Based on this structural information, structure-aware serving improves future requests' prefix locality and give fine-grained protection to future-hot prefixes when ordinary eviction would discard them.

Our contributions are summarized as follows:

- We identify the LLM-centric agent design as an architectural bottleneck and propose structure-semantic separation as a new execution boundary for LLM agents.
- We build MotifAgent, an agent-runtime and serving co-design system that realizes structure-semantic separation. MotifAgent uses motif graphs as executable runtime objects to capture recurring tool-use structure, advance evidence-supported transitions, and scope LLM calls at semantic gaps.
- We design a serving interface and backend mechanisms that make agent structure visible to the LLM serving layer. MotifAgent exposes motif stages, structured prompt prefixes, and downstream-release signals to enable motif-guided lookahead scheduling and fine-grained KV-cache retention and eviction.
- We implement MotifAgent on top of an SGLang-based serving stack and evaluate it on representative agent

scenarios and controlled serving workloads. Our results show its impact on LLM calls, token consumption, goodput and tail latency under queueing pressure, and KV-cache behavior under cache pressure.

2 Background

2.1 LLM Agent Control Loops

LLM agents solve tasks by interleaving model decisions with observations from users, tools, or the environment [8, 14, 28]. A common form is the ReAct-style agent [28], where the model interprets the current context, emits an action, receives the resulting observation, and then decides the next step [28]. Tool use fits naturally into this loop because tools give the agent access to external state and specialized functionality. A tool call also creates a typed boundary: the agent emits a structured action, the environment returns fields, and later operations may consume values from that observation. This design is flexible because the next action can depend on what just happened without requiring a complete workflow to be specified in advance. The trade-off is that the recurring control structure remains embedded in text. Operation order, parameter flow, dependency release, and state checks may recur across executions, but the runtime sees them only as prompt context rather than as executable state.

Several agent and language-model programming systems try to reduce the cost of repeated model calls through tool-use training, reusable prompts, declarative pipelines, structured generation, or workflow-style execution [5, 8, 14, 32]. These systems improve how LLM calls are organized or executed, but the default control loop often remains model-mediated. In this paper, we focus on a complementary boundary: when execution progress is already verifiable from runtime evidence, the runtime should advance it directly rather than asking the LLM to rediscover the same control structure.

2.2 LLM Serving and Prefix Caching

LLM serving systems batch requests to improve accelerator utilization. For decoder-only models, each request has a prefill phase that processes the prompt and constructs KV states, followed by a decode phase that generates output tokens autoregressively. Serving systems such as ORCA [29] improve utilization through iteration-level scheduling and batching, while vLLM [6] improves memory efficiency through PagedAttention and block-level KV-cache management.

Prefix reuse is especially important for multi-call LLM applications. When many requests share prompt prefixes, a serving system can cache KV states for those prefixes and avoid recomputing part of prefill. SGLang [32] targets structured language-model programs and introduces RadixAttention, which stores reusable prefixes in a radix tree and reuses cached KV states across calls. Its scheduler can use radix-aware policies to select waiting requests based

on token-prefix locality. Longest-prefix matching favors requests whose token prefix is already well represented in the cache, while DFS-style traversal favors dense prefix subtrees to improve locality.

These policies operate on tokenized prompt prefixes. This design is safe because cache reuse is determined by exact token-prefix identity. However, token-level cache identity does not expose the agent workflow that produced a request. Semantic labels such as task type, motif stage, dependency frontier, or downstream tool releases are not part of cache identity. As a result, a conventional serving backend can optimize current prefix locality, but it cannot directly reason about whether serving one LLM call will unblock later agent work or whether an inactive prefix is likely to become useful again after a tool gap.

2.3 SLO Goodput for Agent Execution

LLM serving is often measured by throughput and latency, but agent serving also needs task-level metrics. A single user task may contain multiple LLM calls and tool calls. Delaying an intermediate LLM call can block downstream tool calls and extend the top-level task latency, even if token throughput remains high. Therefore, the serving objective for agents is not only to maximize generated tokens per second, but also to complete useful tasks within latency targets.

We use SLO hit rate to denote the fraction of tasks completed before a deadline, and correct-and-SLO goodput to denote the rate of tasks that are both correct and within the SLO. The correctness condition is important because a scheduler should not improve latency by changing task behavior or reducing completion quality. In our setting, the backend only changes service order and cache retention priority. It does not change prompts, tool results, or token-prefix cache identity. This lets us evaluate whether structure-aware serving improves task-level goodput under queueing pressure while preserving the semantic behavior of the agent.

3 Motivation

Building on the separation between agent execution and LLM serving discussed in §1, this section explains why structure-semantic separation should be designed jointly across the agent runtime and the serving backend. Recurring tool-use executions expose reusable structure, but this structure is useful only if the runtime can execute verified progress safely (§3.1), invoke the LLM only for bounded semantic gaps (§3.2), and expose the resulting progress to the backend without changing serving correctness (§3.3).

3.1 Observation 1: Tool-Use Traces Expose Executable Structure

ReAct-style agents are flexible because control can return to the model after every observation. This flexibility, however, also makes the model act as a universal controller. In

recurring tool-use executions, successful traces often follow the same operation order, transfer the same values from tool observations to later tool inputs, check the same preconditions, and release the same downstream actions after the same predecessors finish. A monolithic model-control loop still pays model calls and tokens to rediscover this routine structure during every execution.

This suggests that repeated tool-use behavior should be captured as executable runtime structure rather than prompt history. Prompt skills are too weak because they only describe structure to the model. Static workflows are too rigid because they freeze one path. A motif needs the middle ground: it should encode tool order, parameter flow, guards, and dependencies as runtime state, while still allowing partial execution and recovery when the trace deviates.

Challenge 1: Make structure executable without freezing execution. A motif must be strong enough for the runtime to execute verified tool order, parameter flow, guard checks, and dependency release, but it must not become a rigid workflow. The runtime must support partial execution, re-entry, and recovery when user language, tool observations, or environment state deviate from the common trace.

3.2 Observation 2: Structure Cannot Replace Semantics, But Can Bound It

Structure alone is not sufficient for safe agent execution. User requests may omit slots, refer to entities indirectly, match multiple candidate motifs, or produce tool observations that do not satisfy the expected guard. Blindly following the structure would be unsafe. The question is therefore not whether structure can replace the LLM, but where the boundary between runtime execution and semantic reasoning should be placed.

MotifAgent draws this boundary through two runtime operations. A *structural operation* runs when current evidence is sufficient: it may invoke a known tool node, copy a verified value, check a guard, or release a downstream dependency. A *semantic operation* runs when evidence is insufficient: it asks the LLM to resolve a typed local gap, such as a missing slot, ambiguous entity, unexpected observation, or failed guard. The LLM is no longer asked to control the whole loop; it answers a bounded question using motif state and compact evidence.

Figure 2 confirms this trend across all three workloads: APIBank [8], TauBench [27], and AppWorld [18]. Semantic operations require far fewer input tokens than ReAct-style calls because they contain only motif state and relevant evidence, rather than the full execution history. This supports the runtime principle of MotifAgent: verified progress should be executed structurally, while the LLM should be reserved for local uncertainty that the runtime cannot safely resolve.

Challenge 2: Semantic reasoning with bounds but without full-history control. When structure is insufficient, the system still needs the LLM, but the LLM should not

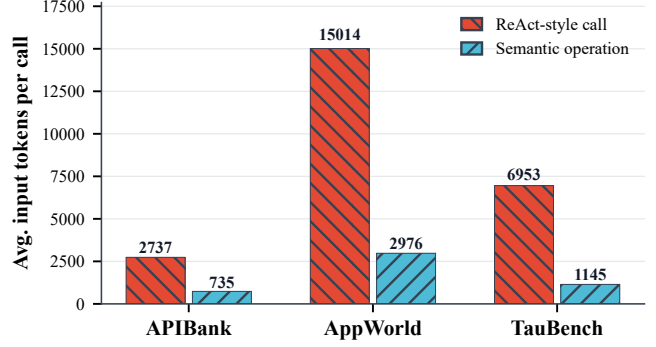


Figure 2. Semantic operations reduce input tokens by using compact motif state and local evidence.

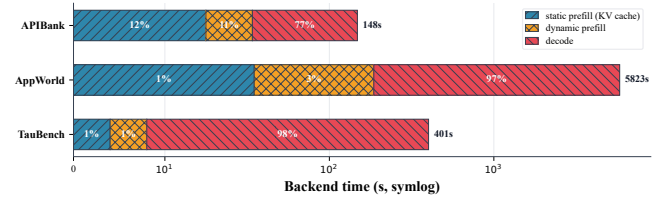


Figure 3. Motif-style execution still leaves decode and queueing as the dominant serving bottlenecks.

resume control of the full agent loop. The runtime must turn uncertainty into typed semantic operations, so the model resolves only the local gap needed to continue execution.

3.3 Observation 3: Bounded Semantic Operations Create a New Serving Workload

Structure-semantic separation changes not only the size of model calls, but also their role in execution. Some calls are short but latency-critical because they release downstream motif stages. Some fallback calls are rare but dominate tail latency. Some stable prefixes are not hot in the current backend queue, but become useful again after a tool gap. The backend therefore no longer serves a homogeneous stream of independent requests; it serves requests whose value depends on motif progress, dependency release, fallback risk, and future prefix reuse.

Figure 3 shows why this becomes a systems problem rather than only a runtime optimization. Even after prompt-side work is reduced, backend time remains dominated by generation and queueing effects. A backend that only optimizes current token-prefix locality may miss which short calls unblock downstream work, which fallback paths are becoming tail-critical, and which prefixes should be retained for reuse after tool gaps.

A natural system-level attempt is to group structured requests before backend admission. This can improve locality when similar motif stages arrive in a dense burst, but it is not robust as the main design. Under less synchronized arrivals,

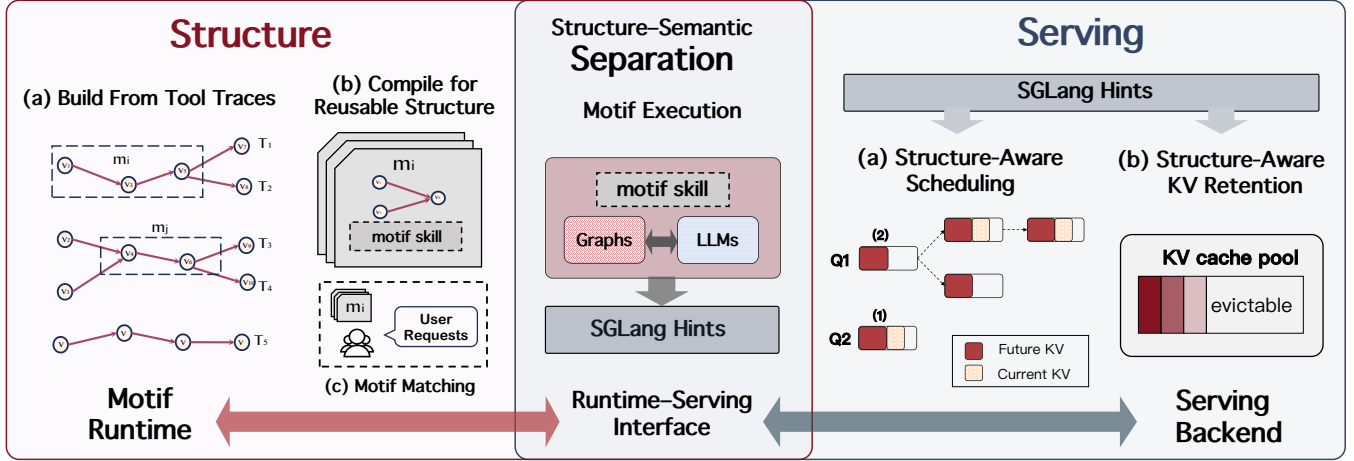


Figure 4. MotifAgent’s system architecture.

front-end grouping can fragment batches, reduce batch fullness, and delay admission. More importantly, the backend already has the real tokenized waiting queue, radix-cache state, and running batch state. Motif information should guide backend decisions as advisory metadata, not force an external order before admission.

Challenge 3: Expose motif progress to the backend without changing serving correctness. Bounded semantic operations create requests whose value depends on execution role, dependency release, fallback risk, and future prefix reuse. The backend should use motif progress to guide scheduling and cache retention, but token-prefix matching must remain the correctness boundary for KV reuse, and backend scheduling must still operate on the real waiting queue and radix-cache state.

3.4 Design Implications

These observations suggest a simple principle: control should be evidence-based. The evolution of motifs is verified in the runtime where tool nodes, parameter-flow edges, guards and dependencies are executed without the need of repeated model control. Unverified uncertainty belongs to bounded semantic mediation, where the LLM resolves only the local gap needed to continue. Serving decisions belong to the backend, where motif progress can guide scheduling and retention, but token-prefix matching remains the correctness boundary for KV reuse.

The resulting design is purposely narrow. MotifAgent does not replace radix scheduling with an external motif batcher, nor does it claim every LLM call should be graph-executed. It keeps verified progress in the structure run at runtime, semantic uncertainty in bounded mediation, token-prefix matching in the backend, and uses motif progress to associate agent execution with serving decisions.

4 MotifAgent Design

This section describes how MotifAgent realizes structure-semantic separation. We first give the architecture and execution bound (§4.1), then define motif skills as evidence-validated operation graphs (§4.2). The runtime executes verified graph progress and invokes scoped semantic mediation at unresolved gaps, while the serving layer consumes advisory motif hints for lookahead scheduling (§4.3) and fine-grained KV-cache retention (§4.4).

4.1 Overview

MotifAgent moves recurring tool-use progress out of the default LLM control loop without removing the model from ambiguous or unexpected states. It has three design goals. First, reusable tool behavior should be represented as executable runtime state, not only as prompt text. Second, the runtime should advance only evidence-supported transitions and should turn missing or ambiguous information into bounded semantic mediation requests. Third, structural progress should be visible to the serving backend as advisory metadata, while exact token-prefix identity remains the correctness boundary for KV-cache reuse.

Architecture. Figure 4 shows the three layers of MotifAgent. The compiler mines recurring tool-use traces into motif skills. At runtime, a matched positive motif is instantiated as a motif instance containing the motif id, current stage, ready frontier, bound values, verified parameter-flow edges, unresolved semantic gaps, downstream-release estimates, and cache-retention ranges. The Structure Engine advances ready graph nodes when the required evidence is present. The Semantic Engine receives typed mediation requests when a slot, candidate, guard, or observation cannot be resolved structurally. The serving backend receives a projection of this state through `sglang_hints`; these hints

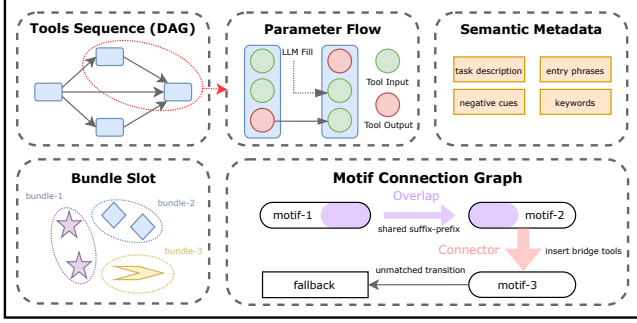


Figure 5. A **motif skill** combines executable graph structure with semantic and transition metadata.

guide scheduling and retention but never alter prompts, tool results, or KV-cache matching.

Execution bound. We model a tool-use execution as structural transitions interleaved with semantic decisions. A structural transition is runtime-verifiable: it may invoke a tool with determined arguments, copy a value from a tool result, check a precondition, update state, or release downstream graph nodes. A semantic decision requires the model because the available evidence is insufficient, for example under missing slots, ambiguous entities, candidate selection, or unexpected observations.

For a task τ , let $\mathcal{D}_\tau = (V, E)$ be its semantic dependency graph. Each node $v \in V$ is a model-resolved semantic decision, and an edge $(u, v) \in E$ means that v depends on evidence produced after u . The semantic critical path is:

$$\text{SCP}(\tau) = \max_{\pi \in \mathcal{D}_\tau} |\pi|. \quad (1)$$

Any execution framework needs at least $\text{SCP}(\tau)$ sequential LLM interaction rounds. A step-by-step agent may instead invoke the LLM once per model-mediated control step. Therefore, the ideal call-round speedup is bounded by:

$$\text{Speedup}_{\max}(\tau) \leq \frac{N_{\text{step}}(\tau)}{\text{SCP}(\tau)}, \quad (2)$$

where $N_{\text{step}}(\tau)$ is the number of model-mediated steps in the baseline loop. MotifAgent approaches this bound only when motif coverage is high and runtime evidence validates the structural transitions; otherwise it falls back to typed mediation or guarded ReAct execution.

4.2 Motif Skills and Operation Graphs

MotifAgent uses two kinds of motif skills. *Positive motifs* encode recurring successful tool-use behavior as executable operation graphs. *Negative motifs* encode repeated terminal failure patterns and are used only as guards during ReAct fallback; they are never executed as plans.

Algorithm 1 summarizes the offline compiler. The deterministic stages normalize traces, mine reusable signatures, bind support slices, and build connector options. The

Algorithm 1 Motif skill compilation

Require: Trajectories $\mathcal{T}$, tool schemas Σ

Ensure: Positive skills $\mathcal{P}$, negative guards $\mathcal{N}$, connector table C

```

1:  $\hat{\mathcal{T}} \leftarrow \text{NORMALIZE}(\mathcal{T}, \Sigma)$ 
2:  $(\mathcal{T}^+, \mathcal{T}^-) \leftarrow \text{SPLITBYOUTCOME}(\hat{\mathcal{T}})$ 
3:  $S \leftarrow \text{MINE\_SIGNATURES}(\mathcal{T}^+)$ 
4:  $C \leftarrow \text{BUILD\_CONNECTORS}(S, \hat{\mathcal{T}})$ 
5:  $\mathcal{P}, \mathcal{N} \leftarrow \emptyset, \emptyset$ 
6: for all  $s \in S$  do
7:    $B \leftarrow \text{BUILD\_SKELETON}(s, \Sigma, C)$ 
8:    $E \leftarrow \text{BIND\_SUPPORT}(s, \mathcal{T}^+)$ 
9:    $A \leftarrow \text{LLM\_COMPILE}(B, E)$ 
10:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{VALIDATE\_REPAIR}(A, E)\}$ 
11: end for
12:  $\mathcal{N} \leftarrow \text{MINE\_FAILURE\_GUARDS}(\mathcal{T}^-)$ 
13: return  $(\mathcal{P}, \mathcal{N}, C)$ 

```

LLM is used only in `LLM_COMPILE` to fill semantic fields and unresolved requirements; `VALIDATE_REPAIR` checks the result against support evidence and demotes unsupported parameter-flow edges before runtime use. Failed or abandoned traces produce negative guards, not executable graphs.

Figure 5 shows the internal structure of a positive motif. The operation graph contains tool nodes, dependency structure, and executable node types such as tool calls, verified extraction, bounded LLM filling, response generation, or mediation handoff. Parameter-flow edges record whether a downstream input is copied from a prior tool output, extracted from local evidence, or left as a semantic requirement. Semantic metadata supports motif retrieval and false-positive filtering, but does not itself execute the workflow. The dialogue contract bounds which missing values may be requested from the user or the LLM; bundleable slots allow related values to be resolved together. Connection hints describe how execution may continue into another motif through suffix-prefix overlap or observed connector tools. These hints guide re-entry and serving lookahead, but they do not override runtime evidence checks.

This representation is stronger than prompt-only skills because the runtime can check guards, propagate values, release ready nodes, and execute tools without asking the LLM to rediscover the workflow. It is less rigid than a static workflow because missing slots, ambiguous candidates, unexpected observations, and fallback branches become typed mediation requests rather than immediate failures. Since motifs are explicit graphs, the runtime can also compose them across a task: for example, it may execute a retrieval motif, re-enter through a connector, and continue with a verification or response motif while preserving local evidence.

4.3 Structural-Aware Scheduling

Structure–semantic execution makes each LLM call more than a token sequence. Besides its prompt, the call carries local workflow context: its motif id, stage id, tool-schema prefix, stable prompt layout, and the downstream calls that may become ready after it finishes. We call the resulting backend policy *Structural-Aware scheduling* (SA): it uses motif context as a conservative lookahead layer on top of radix scheduling, while a conventional radix scheduler only observes current token-prefix locality and cache state.

For each remaining LLM call request, MotifAgent attaches `sglang_hints` to the OpenAI-compatible request. These hints include the current motif and stage, the mapping from tool lists to canonical tool-schema prefixes, the stable prefix ladder, and a bounded set of possible downstream calls. For fallback or typed-mediation calls, each downstream candidate also carries its predicted prefix, confidence, and TTL. The backend uses these hints to estimate which prefixes may appear soon or become useful again. The hints are advisory only: they affect scheduling and eviction priority, but never change request semantics or KV-cache identity.

At each scheduling point, the backend has a waiting queue W , a batch-size limit K , and the current radix-cache state. The base radix policy first proposes a locality-preserving batch B_{base} . SA then considers motif-compatible batches that may have lower immediate locality but higher future locality. For example, two requests Q_1 and Q_2 may currently share the same prefix. Serving Q_2 first may create a longer prefix that will be useful to a later request released by Q_1 ; if that prefix is evicted before the later request arrives, the system loses useful KV reuse. The scheduling problem is therefore a tradeoff between current prefix reuse and predicted future prefix reuse.

SA uses a simple conservative rule. Each candidate batch receives a score consisting of its current prefix reuse plus a weighted future-prefix value estimated from valid downstream hints. The scheduler chooses a motif-aware batch only when its future-prefix value is large enough to compensate for the immediate locality loss relative to B_{base} . Otherwise, it follows the base radix order. If hints are missing, have low confidence, or expire, the future-prefix value becomes zero and SA naturally falls back to ordinary prefix-aware scheduling.

4.4 Fine-Grained KV-Cache Retention

Motif execution also changes cache management. During multi-step agent execution, an LRU-based policy may evict prefixes that will soon be revisited after a tool gap. In contrast, request-level KV-cache pinning may preserve the entire request prefix, including dynamic evidence that is unlikely to be reused. This coarse policy increases cache pressure. Motifs provide a finer-grained signal: they predict which stable

Motif-based LLM requests prompt layout

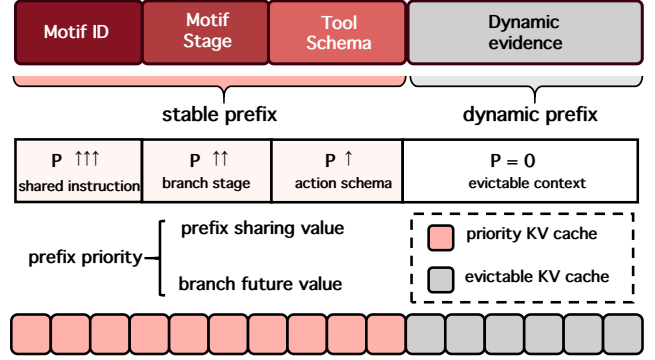


Figure 6. Fine-grained KV-cache retention protects reusable motif prefixes while dynamic evidence remains evictable.

prefixes are likely to be used by upcoming downstream calls and assign higher retention priority only to those ranges.

Figure 6 illustrates the retention policy. A motif-based prompt has a layered layout. Upstream ranges, such as motif id, motif stage, and tool schema, are stable and often shared across requests. Downstream ranges, such as user-specific inputs and tool outputs, are dynamic and usually not reused. This layout gives the backend a natural priority order: stable upstream ranges receive higher retention priority, while dynamic evidence remains evictable.

The backend maintains a shadow tree aligned with the radix tree. The radix tree remains responsible for cache lookup and token-prefix matching. The shadow tree only records motif-provided eviction priorities. When cache pressure requires eviction, the backend evicts low-priority prefix ranges first, instead of relying only on local LRU order or pinning an entire request. If the predicted downstream path does not appear before its TTL expires, the extra priority is removed and the cache falls back to ordinary eviction behavior.

This design keeps the correctness boundary unchanged. KV-cache reuse still depends only on exact token-prefix matching. Motif id, stage id, and downstream predictions only guide scheduling and eviction priority; they never make different token prefixes share KV blocks.

5 Implementation

MotifAgent is implemented as a Python agent runtime integrated with a modified SGLang v0.5.9 backend. The implementation has three layers: an agent layer that extracts and executes motifs, a system interface layer that carries structural metadata to serving, and a backend layer that uses this metadata for scheduling and KV-cache management.

Agent layer. The Python agent runtime builds motif libraries from tool-use traces and records tool operations,

parameter-flow edges, dependency constraints, and runtime-evidence requirements. During execution, the runtime matches the current request and observed environment state against the motif library. The Structure Engine executes ready motif nodes when their required evidence is available, while the Semantic Engine resolves typed mediation requests with schema-constrained LLM outputs when execution reaches a semantic gap. For structured and scoped prompts, the prompt renderer uses the `stage_motif_schema` layout.

System interface layer. The runtime attaches advisory `sclang_hints` to OpenAI-compatible requests. These hints encode motif id, motif stage, prefix ladders, structured prompt segments, downstream-release credit, cache-retention ranges, and priority metadata. The hints are advisory rather than semantic: missing or invalid fields are ignored, and ordinary requests follow standard SGLang behavior.

Backend layer. The modified SGLang backend parses `sclang_hints` into structured request metadata. The scheduler uses motif stage, prefix structure, and downstream-release information for Structural-Aware scheduling (SA). The radix cache uses cache-retention ranges and priority hints to protect KV blocks that are likely to be reused across motif stages or tool gaps, and evicts lower-benefit blocks first under cache pressure. Motif and stage ids are stored only as metadata; token-prefix matching remains the correctness boundary for KV-cache reuse.

6 Evaluation

We organize our evaluation into two parts: generality at the algorithm layer and behavior at the serving layer. We first quantify the runtime effects of structure-semantic separation on three representative tool call benchmarks. Then we replay frozen traces as serving workloads to study how structural information affects backend scheduling and cache behavior in a representative serving scenario.

6.1 Runtime-Layer Setup

Workloads. We use APIBank [8], TauBench [27], and AppWorld [18] to evaluate the agent-runtime layer. They represent single-turn tool use, multi-turn tool use, and web automation tool use, respectively. These workloads challenge whether MotifAgent can reduce the model-control work in different tool-use settings.

Models. The algorithm-layer experiments use DeepSeek-V4-Flash to evaluate task success, LLM calls, token usage, motif execution, and fallback behavior. We use strong model instead of test on local model because AppWorld tasks are dependent on model capability and weak models can hide the runtime effect with task failures.

Baselines. We compare MotifAgent with ReAct [28], ExpeL [31], AWM [21], Agent-KB [16], and APC [30]. ReAct is the most general model-centric agent baseline. The other

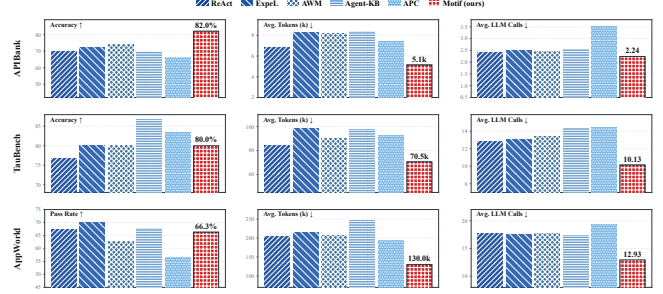


Figure 7. Algorithm-layer results across all benchmark. MotifAgent keeps task success competitive while reducing average token usage and LLM calls across three tool-use workloads.

baselines use experience- or memory-based reuse mechanisms, so they test whether motif skills provide benefits beyond conventional experience reuse.

Metrics. We report task success rate or partial pass rate, the number of LLM calls, and token consumption. These metrics capture agent behavior and deployment cost, and they are determined totally by the agent runtime rather than the backend.

6.2 Runtime-Layer Results

In this experiment, we study whether structure-semantic separation can reduce repeated model-control work across different tool-use workloads.

Figure 7 shows that MotifAgent reduces model-control work across all three workloads. On APIBank, MotifAgent reaches 82.0% accuracy while using 5.1k tokens and 2.24 LLM calls per task. On TauBench, MotifAgent reaches 80.0% accuracy while using 70.5k tokens and 10.13 LLM calls per task. On AppWorld, MotifAgent reaches 66.3% pass rate while using 130.0k tokens and 12.93 LLM calls per task.

The result does not require MotifAgent to win every success metric. The main point is different: MotifAgent keeps task success competitive while reducing repeated model-control work. This supports the core algorithm-layer claim that verified structure can replace many model-mediated control steps, while the LLM remains available for semantic gaps.

6.2.1 AppWorld Difficulty Breakdown. This experiment studies how MotifAgent behaves as task difficulty increases. We group AppWorld tasks into Level 1, Level 2, and Level 3, and report pass rate, average tokens per task, and average LLM calls per task.

Figure 8 shows the expected difficulty trend. As tasks become harder, pass rate drops and both token usage and LLM calls increase. MotifAgent follows the same trend, but it uses fewer tokens and fewer LLM calls than the baselines across difficulty levels. This suggests that motifs do not remove all

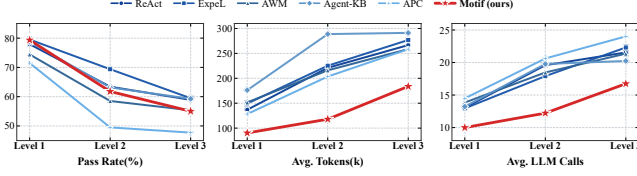


Figure 8. AppWorld results by task difficulty. Harder tasks reduce pass rate and increase model-control work for all methods, but MotifAgent stays on a lower-token and lower-call execution path.

semantic work, but they reduce repeated structural control even when tasks become harder.

This breakdown is used only for algorithm-layer analysis. It shows how structure–semantic separation behaves as task difficulty increases. We do not use task difficulty itself to justify the serving workload. For the serving experiments, we choose AppWorld frozen traces because their generated LLM calls have longer prompts, more stages, and more heterogeneous execution paths, which create stronger queueing and cache pressure for the backend.

6.3 Serving-Layer Setup

Workload. We use APIBank-derived frozen traces as the main serving workload because it provides clear structural signals and fine-grained atomic tool operations. These traces are collected during the algorithm-layer evaluation and preserve the original LLM-call inputs and output-token lengths.

Replay backend. We replay the original prompt content and token lengths, so all backend policies observe the same prefix structure. We enforce every decoding request to decode to its recorded output length so that the decoding latency is reproduced exactly.

Baselines. For Structural-Aware scheduling, we compare against four backend schedulers. LPM batches requests by the longest reusable prefix, and DFS-weight prioritizes heavily shared branches in the prefix tree [32]. They both use token-prefix locality, but do not use future request information. We also compare with Autellix-PLAS [10], which prioritizes LLM calls from programs that have received less completed service. This tests whether finer-grained motif-stage signals provide benefits beyond prefix locality and program-level attained service. For fine-grained KV retention, we compare against LRU and a Continuum-style request-level KV-cache pinning method [7].

Metrics. Metrics are also two fold. For scheduling, we report scheduling goodput and median request latency. We also use latency ECDFs to compare the full request-latency distribution across policies. For KV cache retention, we report cache goodput and the early-eviction rate of reusable KV blocks.

Load protocol. Requests arrive according to a materialized Poisson arrival trace. We use the same arrival trace for

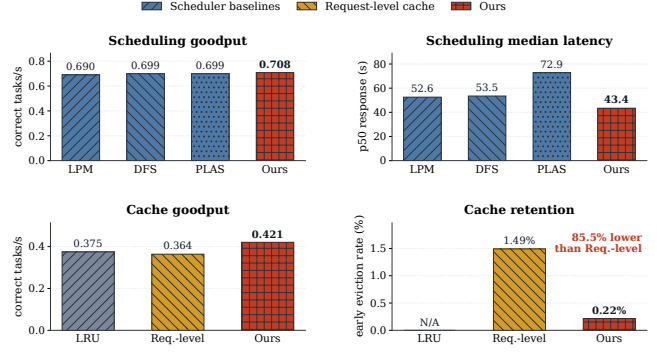


Figure 9. Overall Performance on APIBank.

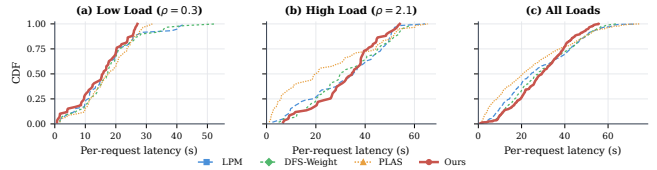


Figure 10. ECDF analysis on AppWorld.

all methods and multiply its arrival timestamps by a fixed time scale. In the APIBank experiment, we use 100 tasks to set the arrival time scale to 13.2. This gives an effective arrival rate of about 3.7 requests/s over the evaluated task window.

6.4 Serving-Layer Results

We first use APIBank to isolate how backend-visible structure affects serving behavior. It contains dense and regular motif reuse.

For scheduling, MotifAgent mainly reduces latency while preserving goodput. LPM and DFS leverage existing prefix locality but don’t know which calls lead to a motif or unlock downstream stages. PLAS considers progress at the program level, but its greedy choices can break prefix locality. MotifAgent uses the predicted downstream motif path to favor calls that can create more future prefix reuse, which lowers median latency without sacrificing goodput.

The cache results show a complementary benefit. Request-level retention protects whole prefixes and can waste cache space under pressure. MotifAgent instead protects reusable prompt regions predicted by motif structure. This fine-grained retention improves goodput and sharply reduces early eviction of blocks that will soon be reused. The result supports a key serving insight: structural information is useful not only for scheduling requests, but also for deciding which parts of the KV cache are worth keeping.

Figure 10 provides a broader validation on AppWorld. The ECDFs show the same qualitative trend in a more heterogeneous workload. The structure-informed policy does not

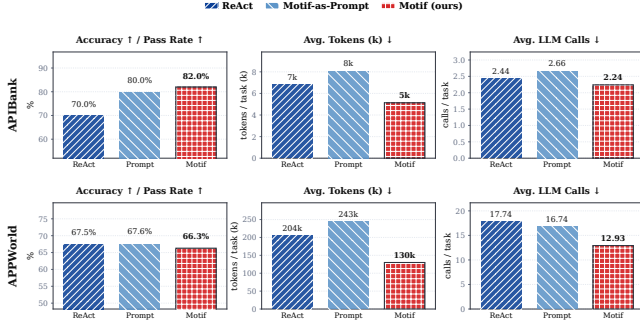


Figure 11. Motif-as-prompt ablation on APIBank and AppWorld. Motif-as-prompt uses the same compiled motif library and matcher as MotifAgent, but only renders the matched motif as a prompt hint for a ReAct agent.

always minimize the median, but it gives a more stable latency distribution and avoids the long-tail behavior left by prefix-only or program-level policies. This suggests that motif progress is most useful for controlling tail behavior and deadline risk, rather than merely optimizing average latency.

6.5 Ablation Study

6.5.1 Motif as Prompt. Existing experience-enhanced and skill-based agents mainly reuse past behavior as textual guidance. MotifAgent introduces structural experience: recurring tool-use graphs compiled into executable motif skills. These skills allow graphs to participate in agent decision-making, rather than merely describe a workflow.

To test this design, we compare MotifAgent with a motif-as-prompt variant. It uses the same motif library and matcher, but only adds a workflow hint to the system prompt and leaves all decisions to the ReAct agent.

Figure 11 shows that although motif skills are useful as prior knowledge, it does not provide the full benefit. On APIBank, motif-as-prompt improves accuracy from 70.0% to 80.0%, but it also increases average tokens from 7k to 8k and LLM calls from 2.44 to 2.66. In contrast, MotifAgent reaches 82.0% accuracy while reducing tokens to 5k and LLM calls to 2.24.

On AppWorld, motif-as-prompt has almost the same pass rate as ReAct, 67.6% versus 67.5%, but increases average tokens from 204k to 243k. MotifAgent achieves a comparable pass rate of 66.3%, while reducing average tokens to 130k and LLM calls from 17.74 to 12.93.

This confirms that the main benefit of MotifAgent does not come from telling the model a workflow in text. It comes from executing reusable structure in the runtime, tracking parameter dependencies, and falling back only when semantic mediation is needed. In other words, MotifAgent **makes graphs active decision structures** in the agent runtime.

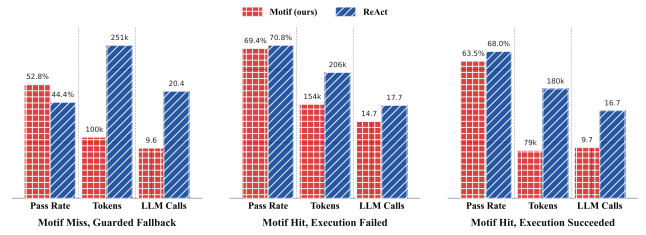


Figure 12. AppWorld runtime behavior by motif outcome. MotifAgent reduces model work not only when motifs execute successfully, but also when guarded fallback bounds unproductive execution or when semantic operations are constrained by matched structure.

6.5.2 Motif Behavior Analysis. To further explain where MotifAgent gains come from, we break down AppWorld executions by different motif operation. We consider three cases: no progress motif is matched and the runtime enters guarded fallback; motif skill is matched but execution partially falls back to semantic operations; and a motif is matched and executed successfully.

Figure 12 shows that MotifAgent still helps when motifs are not fully executable. A common failure mode in agent execution is local error loops. During motif compilation, **repeated tail failures** can be turned into guarded fallback motifs, which stop unproductive execution when no progress motif is matched. In the motif-miss group, these guards improve pass rate while cutting both token usage and LLM calls by more than half.

When a matched motif cannot complete the task on its own, MotifAgent still preserves task success while reducing model work. Its pass rate remains close to ReAct, 69.4% versus 70.8%, and token usage drops from 206k to 154k. This shows that even **partial structural execution** is useful: the LLM still resolves semantic gaps, the matched motif constrains what it needs to decide.

The largest efficiency gain appears when motif execution succeeds. In this group, MotifAgent reduces token usage from 180k to 79k and LLM calls from 16.7 to 9.7. Although its pass rate is slightly lower than ReAct, this result supports the main runtime claim: executable structure can **replace** many model-mediated control steps and lower the serving cost of agent execution.

6.5.3 End-to-End Integration and Backend Ablations. After showing that motif execution reduces model-control work, we next ask whether this reduction translates into real agent-serving gains and how much each backend mechanism contributes.

Figure 13 answers this question on APIBank. Panel A shows the end-to-end effect: MotifAgent reduces p95 latency from 580s to 124s, improves correct goodput from 0.109 to 0.405 tasks/s, and cuts average prompt tokens from 6.7k to

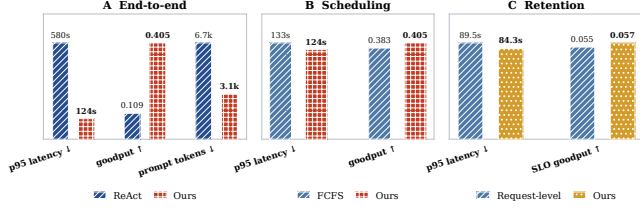


Figure 13. End-to-end integration and backend ablations on APIBank.

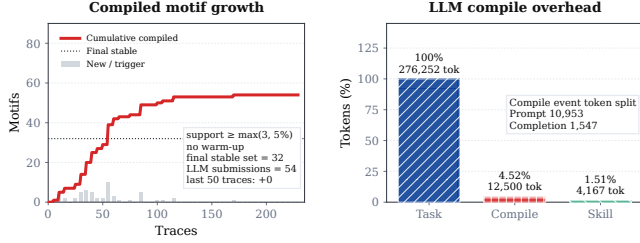


Figure 14. Motif compilation growth and token overhead on AppWorld.

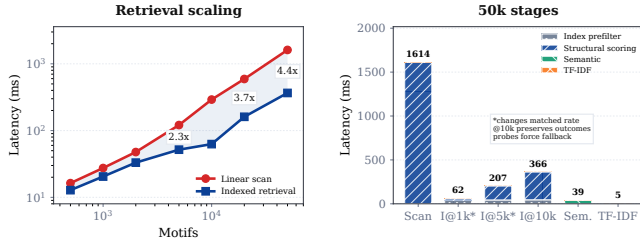


Figure 15. Motif retrieval overhead and scalability on AppWorld.

3.1k. This confirms that executable motifs reduce model-side work and improve serving behavior.

Panels B and C isolate the backend effects. Motif-aware scheduling gives a smaller but consistent gain over baseline, reducing p95 latency from 133s to 124s and improving goodput from 0.383 to 0.405 tasks/s. Layered KV-cache retention improves over request-level retention by protecting reusable prompt regions rather than whole request prefixes, reducing p95 latency from 89.5s to 84.3s and avoiding premature eviction of protected blocks.

Together, these results suggest that the main gain comes from the runtime motif execution, while the backend policies provide complementary benefits once the structure is exposed to the serving layer.

6.6 Overhead and Scalability Analysis

Compilation overhead. Figure 14 shows that motif compilation grows slowly as more successful AppWorld traces are observed. Under shuffled trace order, the final stable library contains 32 motif signatures, and MotifAgent submits

54 candidates to the LLM skill compiler. The last 50 traces introduce no new compile candidate. This shows that motif discovery becomes sparse after common structures have been covered.

The LLM compile cost is also small after reuse. In a live compile probe, one compile event produces three new skills and uses 12.5k tokens in total. This is 4.52% of the average token consumption of one AppWorld task, or 1.51% per new skill. Thus, compilation is not free, but its LLM cost is paid only for newly discovered motif signatures and is amortized by later reuse.

Retrieval scalability. Figure 15 shows the retrieval cost as the motif library scales. Linear scan becomes expensive at large library sizes and reaches 1.61s per query at 50k motifs. The main cost comes from structural candidate scoring, rather than semantic or TF-IDF fallback.

Indexed retrieval reduces this cost. At 50k motifs, the recall-preserving indexed variant reduces latency to 366ms per query, giving a 4.4 $\times$ speedup over linear scan. This shows that motif retrieval remains scalable with a structural index. It also suggests that semantic and TF-IDF stages are useful as fallbacks, but they should not replace structural retrieval because structural matching preserves tool-chain and coverage constraints.

7 Discussion and Limitations

7.1 Why This Is a Systems Problem

Structure-semantic separation brings new opportunities to the system design. The runtime needs explicit graph state, typed mediation contracts, evidence tracking, and recovery boundaries. Once those mechanisms reduce calls and tokens, the backend is no longer optimizing a large uniform pool of long prompts. It is deciding which small structured stages should consume scarce service slots under overload. That decision affects whether a workflow releases downstream work, whether future-hot prefixes survive a tool gap, and whether one agent class accumulates tail lag while another gets locality preference.

7.2 When MotifAgent Helps

MotifAgent is most likely to help for tool-use agents with repeated tool patterns, stable schemas, verifiable parameter flow, queueing pressure, heterogeneous workflow value, repeated structural prefixes, and enough reuse distance or cache pressure for retention to matter. It is less likely to help under low load, when all requests enter service immediately; when the prompt is too short for prefill locality to matter; when there are no stable tool-feedback boundaries; or when completion quality is dominated by motif matching/fallback failures rather than serving order.

8 Related Work

8.1 LLM Agents and Reusable Tool-Use Structure

LLM agents enable language models to reason, take actions, observe external state, and invoke tools [20, 24]. ReAct-style agents return control to the model after each observation [28], whereas Toolformer, ToolLLM, and API-Bank study how models learn, choose, and evaluate API-based tool usage [8, 13, 14]. Benchmarks like WebArena, τ -bench, AppWorld, OSWorld, and SWE-agent demonstrate that realistic agents operate over long-horizon, stateful environments with external tools, user interactions, and executable side effects [18, 25–27, 34]. The works above motivate the development of tool-use agents, but also reveal the cost of repeatedly routing execution progress through the model.

Another line of work reuses agents’ past experience. ExpeL extracts natural-language insights from trajectories [31], AWM stores reusable workflows in agent memory [21], AgentKB retrieves cross-domain workflows and diagnostic fixes [16], and APC adapts cached plans for similar tasks [30]. Agent programming systems further provide a more structured way to build LLM applications via modules, workflows, graphs, memory, or multi-agent orchestration, such as DSPy [5], SGLang [32], AutoGen [23], Voyager [19], and Language Agent Tree Search [33]. These systems primarily reuse or organize structure as prompt guidance, retrieved memory, search state, conversation logic, or developer-specified control around an LLM controller. This makes structure useful to the model, but the model is still usually responsible for interpreting that structure and deciding how execution should proceed. Instead, MotifAgent instantiates matched motifs as executable runtime state. Tool order, parameter flow, guards, and dependencies are checked against current evidence, with deviations leading to guarded fallbacks or typed local mediation, rather than direct plan replay.

8.2 LLM Serving and KV-Cache Management

Through batching, scheduling, prefix reuse, and memory management, LLM serving systems optimize throughput and latency. ORCA proposes iteration-level scheduling [29]. vLLM uses PagedAttention to manage the KV-cache memory efficiently [6]. SARATHI combines chunked prefills with decode-maximal batching [1]. FastServe uses token-level preemptive scheduling [22]. SGLang uses RadixAttention to reuse KV cache across shared prompt prefixes [32]. Autellix PLAS prefers LLM calls with program-level attained service [10]. Such systems optimize requests, tokens, prefixes, memory blocks, or program-level progress.

Another line of KV-cache work studies where cache is stored, represented, or transferred. vAttention optimizes dynamic KV-cache management [11]. FlexGen leverages

GPU, CPU, and disk resources for high-throughput inference with limited accelerator memory [15]. CacheGen compresses and streams KV cache [9]. Mooncake proposes a KV-cache-centric disaggregated architecture [12]. These systems mainly reason about the cache as a memory-management resource. MotifAgent is complementary, exposing motif stage, structured prompt prefixes, downstream-release hints, and cache-retention ranges as advisory serving metadata. These hints enable the backend to reason about which calls might release downstream work, and which prompt regions might be reused after tool gaps, but request semantics and KV-cache correctness still only depend on exact token-prefix identity.

8.3 Workflow Scheduling

Classical workflow and DAG schedulers use dependency structure, critical paths, and resource estimates to schedule dependent tasks. List scheduling and critical-path heuristics are foundational techniques for precedence-constrained tasks [3, 4]. HEFT schedules DAG tasks on heterogeneous processors [17]. Pegasus [2] automates dependency-aware execution, data movement, and resource management for scientific workflows. MotifAgent borrows the intuition that upstream work can be valuable because it releases downstream operations. However, LLM-agent serving differs from classical workflow scheduling because model requests also interact through token-prefix locality, radix-cache state, KV-cache pressure, task-level SLOs, and runtime semantic gaps. In this setting, a dependency edge is not the only scheduling signal because a request may also be valuable due to prefix reuse or future KV-cache locality. Motif-guided scheduling therefore does not replace backend prefix-aware scheduling with an external workflow scheduler. It provides conservative workflow hints to the serving backend, which still schedules over the real waiting queue and preserves the existing cache-correctness boundary.

9 Conclusion

MotifAgent rethinks LLM agent architecture by separating structural control from semantic judgment. It makes recurring execution progress explicit and executable through motif operation graphs, keeps the agent recoverable through typed mediation when structure is incomplete, and exposes motif progress to the backend through advisory serving hints. This agent-runtime and serving co-design reduces repeated model-control work without removing the LLM’s role in semantic uncertainty. Our evaluation shows that making agent structure both executable and serving-visible can reduce LLM calls and token consumption, improve SLO-oriented behavior under load, and protect reusable KV-cache prefixes under cache pressure. We believe that MotifAgent points out a promising direction for efficient LLM agents.

References

- [1] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [2] Ewa Deelman, Karan Vahi, Mats Rynge, Gideon Juve, Rajiv Mayani, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. 2015. Pegasus, a Workflow Management System for Science Automation. *Future Generation Computer Systems* 46 (2015), 17–35.
- [3] Ronald L. Graham. 1966. Bounds for Certain Multiprocessing Anomalies. *Bell System Technical Journal* 45, 9 (1966), 1563–1581.
- [4] James E. Kelley and Morgan R. Walker. 1959. Critical-Path Planning and Scheduling. *Proceedings of the Eastern Joint Computer Conference* (1959), 160–173.
- [5] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *The Twelfth International Conference on Learning Representations*. <https://doi.org/10.48550/arXiv.2310.03714>
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [7] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and robust multi-turn llm agent scheduling with kv cache time-to-live. *arXiv preprint arXiv:2511.02230* (2025).
- [8] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116.
- [9] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*.
- [10] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025). <https://arxiv.org/abs/2502.13965>
- [11] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2024. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. *arXiv preprint arXiv:2405.04437* (2024).
- [12] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079* (2024).
- [13] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023).
- [14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [15] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning*.
- [16] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. 2025. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229* (2025).
- [17] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274.
- [18] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [19] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).
- [20] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- [21] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. *arXiv preprint arXiv:2409.07429* (2024).
- [22] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2023. Fast Distributed Inference Serving for Large Language Models. *arXiv preprint arXiv:2305.05920* (2023).
- [23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkan Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155* (2023). <https://doi.org/10.48550/arXiv.2308.08155>
- [24] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wensen Cheng, Qi Zhang, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, and Tao Gui. 2023. The Rise and Potential of Large Language Model Based Agents: A Survey. *arXiv preprint arXiv:2309.07864* (2023).
- [25] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*, Vol. 37.
- [26] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37.
- [27] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045* (2024).

- [28] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [29] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538.
- [30] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2026. Agentic plan caching: Test-time memory for fast and cost-efficient llm agents. *Advances in Neural Information Processing Systems* 38 (2026), 103270–103296.
- [31] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642.
- [32] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, Vol. 37.
- [33] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2024. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models. In *Proceedings of the 41st International Conference on Machine Learning*.
- [34] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854* (2023).